

Documentatie PPD

RAPORT TEHNIC

Sistem Concurent de Rezervări pentru Ateliere Culturale

1. Arhitectura Aplicației

Sistemul a fost proiectat pe o arhitectură distribuită de tip **Client-Server**, utilizând un stil arhitectural **RESTful** pentru comunicare. Aplicația este divizată în două componente distincte, decuplate logic:

1. Serverul (Backend):

- Implementat în limbajul **C++** pentru performanță ridicată.
- Utilizează micro-framework-ul **Crow** pentru gestionarea cererilor HTTP și rutare.
- Acționează ca un punct central de procesare, primind cereri JSON (POST `/rezerva`, `/plateste`, `/anuleaza`) și returnând răspunsuri asincrone.
- Stratul de persistență este asigurat de **SQLite3**, configurat în modul **WAL (Write-Ahead Logging)** pentru a optimiza accesul concurent.

2. Clientul (Frontend):

- Implementat folosind biblioteca **React.js** și utilitarul de build **Vite**.
- Interfața este reactivă, oferind feedback vizual utilizatorului (ex: cronometru pentru expirarea plății, notificări de eroare).
- Comunică cu serverul prin apeluri asincrone (`axios` / `fetch`).

Fluxul de date:

Clientul trimite un obiect JSON -> Serverul Crow preia cererea -> Serverul plasează task-ul în Thread Pool -> Un Worker procesează logica de business (Service) -> Acces Bază de Date -> Răspunsul este trimis înapoi prin Future.

2. Decizii de Sincronizare

Într-un mediu multithreaded, accesul simultan la resurse partajate (în acest caz, baza de date SQLite) poate duce la **Race Conditions** (ex: doi clienți rezervă ultimul loc disponibil simultan) și coruperea datelor.

Pentru a asigura integritatea datelor, am implementat următoarele mecanisme:

- **Mutex Global pe Baza de Date (`std::mutex`):**
 - Am optat pentru o strategie de sincronizare "coarse-grained". Clasa `Database` deține un `std::mutex`.
 - În clasa `WorkshopService`, orice metodă care interacționează cu baza de date (`proceseazaRezervare`, `proceseazaPlata`, `runPeriodicChecks`) utilizează `std::lock_guard<std::mutex>`.
 - **Justificare:** Această abordare (RAII) garantează că lacătul (lock) este eliberat automat la ieșirea din funcție, prevenind **Deadlock-urile**. Deși reduce ușor paralelismul la nivel de scriere în DB, garantează 100% consistența tranzacțională, ceea ce este critic pentru un sistem de rezervări.
 - **Thread-ul de Audit (Background Worker):**
 - Acest thread rulează independent și concurează pentru același mutex cu workerii din Thread Pool.
 - Sincronizarea asigură că auditul (verificarea suprapunerilor și ștergerea rezervărilor expirate) nu se execută niciodată simultan cu o tranzacție de plată sau rezervare, oferind o vedere consistentă ("snapshot") a sistemului.
-

3. Utilizarea Futures, Promises și Thread Pool

Pentru a gestiona eficient execuția concurentă și pentru a preveni blocarea serverului la fiecare cerere, am implementat mecanisme avansate de C++:

A. Thread Pool (Implementare Custom)

- Crearea și distrugerea firelor de execuție pentru fiecare cerere (modelul *thread-per-request*) este costisitoare pentru sistemul de operare.
- Am implementat o clasă `ThreadPool` care inițializează un număr fix de fire de execuție (**P=8** în testele noastre) la pornire.

- Acești "workeri" așteaptă într-o buclă infinită, monitorizând o coadă de task-uri (`std::queue`) protejată de un mutex propriu și o variabilă condițională (`std::condition_variable`).

B. Futures și Promises

- Am folosit `std::future` pentru a decupla primirea cererii de procesarea ei.
- Când serverul primește un request HTTP, acesta nu execută logica imediat. În schimb:
 1. Împachetează logica într-un task asincron (`std::packaged_task`).
 2. Adaugă task-ul în `ThreadPool` prin funcția `enqueue` .
 3. Primește înapoi un `std::future` .
- Serverul așteaptă rezultatul prin `future.get()` doar în momentul în care trebuie să construiască răspunsul HTTP. Acest mecanism permite gestionarea erorilor și returnarea rezultatelor din thread-urile worker către thread-ul principal.

4. Analiza Performanței

Pentru evaluare, s-a utilizat un scenariu de testare automată cu următorii parametri:

- **Volum:** 20 de clienți concurenți.
- **Frecvență:** 1 cerere la fiecare 2 secunde per client.
- **Durată:** 120 de secunde.
- **Configurație Server:** 8 Thread-uri.

A. Metrice Observate (Timpuri și Throughput)

Metrică	Valoare Observată
Timp Mediu de Răspuns (Latency)	< 10ms (pentru operații reușite)
Throughput (Cereri/Sec)	~10 cereri/sec (limitat de clientul de test, nu de server)
Grad de Ocupare CPU	Distribuit eficient pe cele 8 thread-uri.

B. Comparatie: Versiune Serială vs. Concurentă

Deși implementarea finală este concurentă, putem compara comportamentul acesteia (8 threads) cu o rulare limitată la 1 thread (echivalentul versiunii seriale), pe baza datelor teoretice și a logurilor.

Caracteristică	Versiune Serială (1 Thread)	Versiune Concurentă (8 Threads)
Gestionarea Cozii	Cererile se execută strict una după alta. Dacă o scriere în DB durează 50ms, a 20-a cerere așteaptă 1 secundă.	Cererile sunt preluate de primul worker liber. Pregătirea datelor și procesarea HTTP se fac în paralel.
Timp de Răspuns (Latenta)	Crește liniar cu numărul de utilizatori simultani.	Rămâne constant și scăzut până la saturarea tuturor nucleelor CPU sau a I/O-ului.
Utilizare Resurse	Ineficientă (un singur nucleu utilizat).	Eficientă (sarcina distribuită pe nuclee).
Experiența Utilizatorului	Blocaje vizibile la trafic intens.	Răspuns rapid, interfață fluidă.

C. Impactul Intervalului de Audit (5s vs 10s)

Din analiza logurilor generate (raport_audit_5s.txt vs raport_audit_10s.txt):

1. **Audit la 5 secunde:**
 - **Avantaj:** Rezervările neplătite sunt eliberate foarte rapid (maxim 5 secunde după expirarea timpului de 12s). Disponibilitatea locurilor crește.
 - **Cost:** Overhead mai mare de sincronizare (blocarea mutex-ului DB la fiecare 5 secunde).
2. **Audit la 10 secunde:**
 - **Avantaj:** Performanță brută mai bună a serverului, deoarece întreruperile pentru audit sunt mai rare.
 - **Dezavantaj:** Un loc poate rămâne blocat "inutil" până la 9 secunde după expirarea reală, reducând șansa altui client de a-l rezerva imediat.

Concluzie:

Arhitectura concurentă cu Thread Pool și Mutex pentru protecția datelor s-a dovedit a fi soluția optimă, asigurând atât **corectitudinea datelor** (zero suprapuneri în loguri), cât și un **timp de răspuns minim** pentru utilizatori.